UNIVERSIDAD DEL NORTE

Estructura de Computadores I – Reto 2 (EC1-2026-30)

Nios II Assembly Language Calculator

With an interactive LED memory game

User Manual and Technical Report

Plataforma: Intel Nios II en DE2-115 (simulador CPUlator)

Profesora: Dr.-Ing. Diana Rueda-Delgado

Autores: [Daniel Garcia], [Victor Mendoza Pestana], [Jesus Contreras]

Barranquilla, Colombia – 2026

SUMMARY

This document describes the design, implementation, operation, and validation of an interactive calculator written entirely in Nios II assembly language and executed on the CPUlator simulator (DE2-115 system). The program communicates with the user exclusively via the memory-mapped JTAG UART, accepts hexadecimal operands in the 0–15 range, and implements addition, subtraction, multiplication, integer division with remainder, and evaluation of the quadratic polynomial $F(x) = ax^2 + bx + c$. A sixth module—a sequence memory game—uses the red LEDs (LEDR) to display a pseudo-random pattern that the user must reproduce. Operations lacking direct hardware support (multiplication and division) are performed using repeated addition and subtraction, and all input/output is handled by polling the peripheral's status registers. The report follows a manual-style format: it first presents the operating procedure for the end-user, then describes each code block and its machine-level rationale, and finally documents a testing campaign using analytically derived expected values.

Keywords— Assembly language, *Nios II*, *JTAG UART*, *memory-mapped I/O*, *polling*, *CPUlator*, *DE2-115*, *integer arithmetic*.

Table of Contents

Abstract..... 2
Table of Contents..... 3
1. Introduction..... 5
1.1 Purpose and scope..... 5
1.2 Functional summary..... 5
1.3 Design features..... 5
2. System overview..... 6
2.1 Hardware platform and memory map..... 6
2.2 Operating principle..... 6
3. User guide..... 9
3.1 Requirements..... 9
3.2 Loading and starting the program..... 9
3.3 Input conventions..... 9
3.4 Usage of each operation..... 9
Operations 1 to 3 (addition, subtraction, and multiplication)..... 9
Operation 4 (integer division)..... 10
Operation 5 (quadratic function)..... 10
Operation 6 (memory game)..... 10
3.5 Ending the session..... 11
3.6 Error messages and troubleshooting..... 11
4. Technical description of the code..... 12
4.1 Constants and assembler directives..... 12
4.2 Startup code..... 12
4.3 Main routine and menu..... 12
4.4 Arithmetic modules..... 13
4.4.1 Addition and subtraction (op_suma, op_resta)..... 13
4.4.2 Multiplication (op_mult, multiplicar)..... 13
4.4.3 Integer division (op_div)..... 14
4.4.4 Quadratic function (op_fx)..... 14
4.5 Input and output subroutines..... 14
4.5.1 putc and read_char (hardware layer)..... 14
4.5.2 print_string..... 15
4.5.3 read_value..... 15
4.5.4 pedir_ab..... 15

4.5.5 print_dec, print_udec, and print_sdec 15
4.5.6 delay and delay_custom.....16
4.5.7 preguntar_continuar.....16
4.6 Memory game (op_juego).....16
4.6.1 Sequence generation.....16
4.6.2 LED display.....16
4.6.3 Response verification.....17
4.7 Data section.....17
4.8 Register usage and calling convention.....17
5. Testing and performance evidence.....18
5.1 Methodology.....18
5.2 Test cases with evidence.....18
5.3 Additional boundary values.....18
5.4 Performance analysis.....19
6. Limitations and future work.....19
7. Conclusions.....20
References 2
1

1. Introduction

1.1 Purpose and scope

The objective is to design and implement a calculator in Nios II assembly language that interacts with the user via a UART terminal, in accordance with the specifications for the Computer Architecture I Exam 2. Upon startup, the system displays a welcome message; the user selects an operation, and the program prompts for the necessary operands (two for binary operations, four for the polynomial), introduces a brief delay to facilitate viewing, and prints the result to the terminal. Nios II is a 32-bit RISC soft processor featuring a load/store architecture and 32 general-purpose registers. The deliverables consist of this report, the assembly source file (LEDS.s) compatible with CPUlator, and a demonstration video.

1.2 Resumen funcional

Table 1. Operations supported by the program.

Option	Module	Description
1	Addition	R = A + B, with A and B ∈ [0, 15]
2	Subtraction	R = A − B; negative results are displayed with a sign
3	Multiplication	R = A × B via repeated addition (without MUL instruction)
4	Integer division	Quotient and remainder via repeated subtraction; division by zero handled
5	Quadratic function	$F(x) = a \cdot x^2 + b \cdot x + c$ con a, b, c, x ∈ [0, 15]
6	Memory game	Sequence of five LEDs on LEDR; response entered by specifying positions 1–5
0	Exit	Close calculator

1.3 Design features

- Operand input accepts digits 0–9 and letters A–F (uppercase or lowercase), covering the 0–15 range with a single keystroke and without requiring an Enter key.
- Any invalid input is rejected with an explanatory message, and the prompt is repeated; the program never terminates due to erroneous data.
- After each operation, the user is asked whether to continue (Y) or exit (N), allowing for an unlimited number of operations within a single execution.
- Subroutines follow a calling convention consistent with explicit stack frames, making them reusable and traceable.

2. System overview

2.1 Hardware platform and memory map

The program targets the Nios II system on the DE2-115 board as modeled by CPUlator. Two peripherals are used, accessed via memory-mapped I/O using the `ldwio` and `stwio` instructions, which bypass the data cache.

Table 2. Memory mapped resources used by the program.

Peripheral / Register	Direction	Program Usage
JTAG UART – DATA	0x10001000	Write: byte to transmit (bits 7:0). Read: received byte (bits 7:0) and RVALID flag (bit 15)
JTAG UART – CONTROL	0x10001004	Bits 31:16 (WSPACE) indicate available space in the transmit FIFO
LEDR (LEDs rojos)	0x10000000	Bits 0–4 control LED0–LED4 on the board
Tope de pila	0x00100000	Initial value of sp (STACK_TOP)

2.2 Principio de operación

Como el programa usa sondeo en lugar de interrupciones, cada primitiva de E/S espera activamente un bit de estado. Este enfoque mantiene el flujo de control determinista y evita configurar el vector de excepciones y los registros ienable/status.

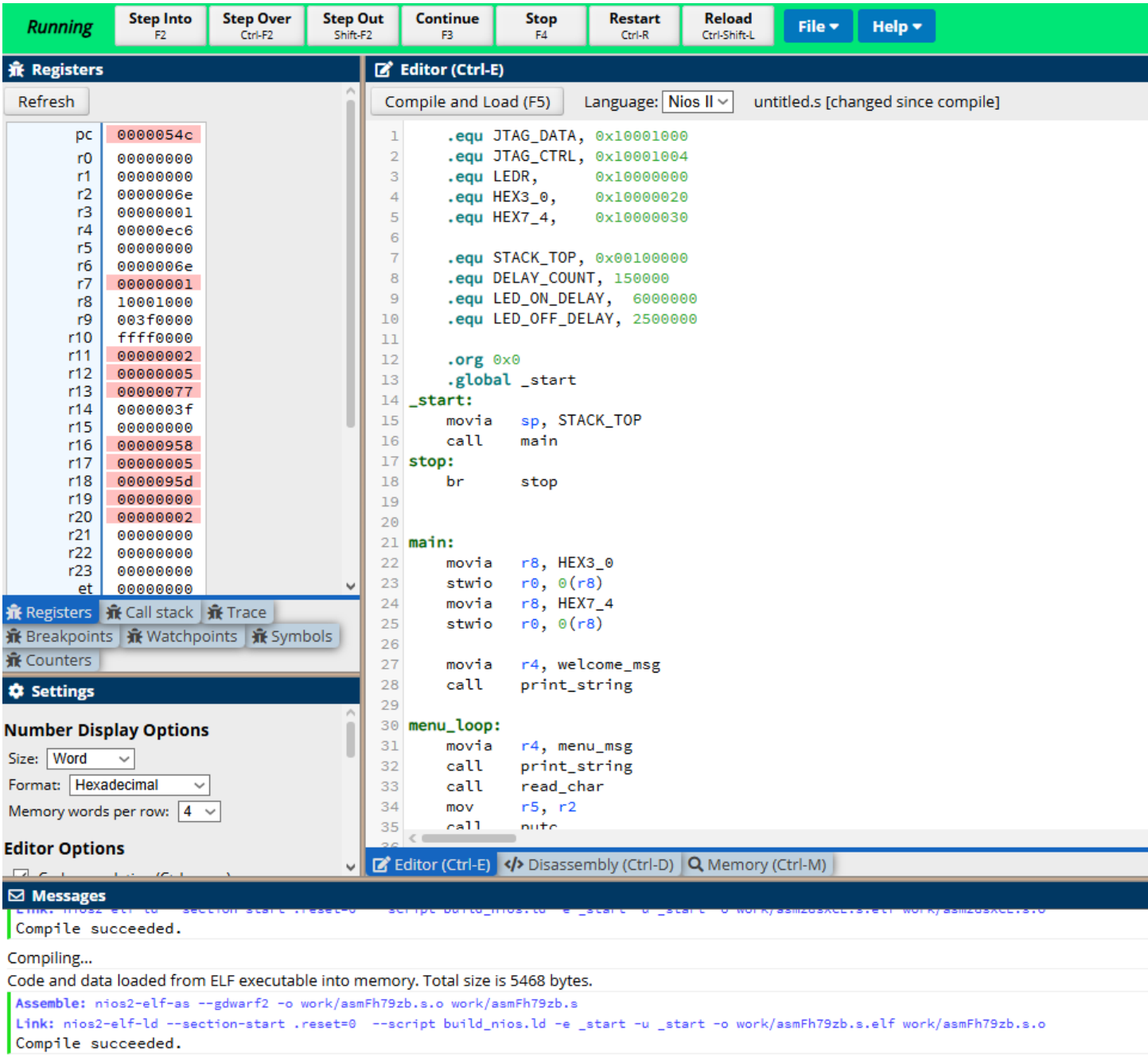


Figure 1. CPUlator configuration, successful compilation, and program startup screen.

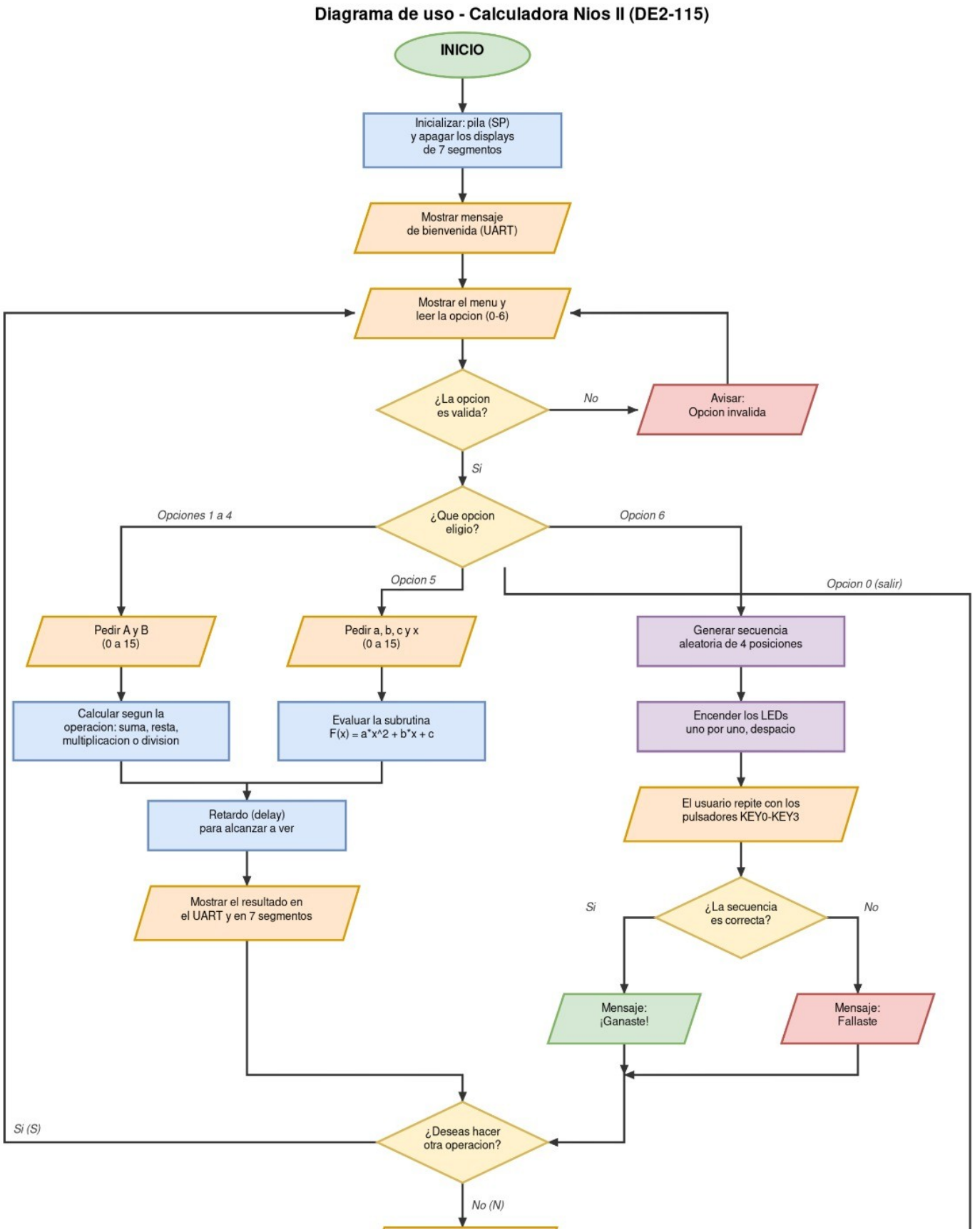

Diagrama de uso - Calculadora Nios II (DE2-115)
INICIO
Inicializar: pila (SP) y apagar los displays de 7 segmentos
Mostrar mensaje de bienvenida (UART)
Mostrar el menu y leer la opcion (0-6)
¿La opcion es valida?
No
Avisar: Opcion invalida
Si
¿Que opcion eligio?
Opciones 1 a 4
Opcion 5
Opcion 6
Opcion 0 (salir)
Pedir A y B (0 a 15)
Pedir a, b, c y x (0 a 15)
Generar secuencia aleatoria de 4 posiciones
Calcular segun la operacion: suma, resta, multiplicacion o division
Evaluar la subrutina F(x) = a*x^2 + b*x + c
Encender los LEDs uno por uno, despacio
Retardo (delay) para alcanzar a ver
El usuario repite con los pulsadores KEY0-KEY3
Mostrar el resultado en el UART y en 7 segmentos
Si
¿La secuencia es correcta?
No
Mensaje: ¡Ganaste!
Mensaje: Fallaste
Si (S)
¿Deseas hacer otra operacion?
No (N)

Figure 2. General program flow: from _start to the menu loop and the exit condition.



Figure 3. Call hierarchy among subroutines (arrows indicate "calls").

3. User Guide

This section allows any user to run the program without assistance from the authors.

3.1 Requirements

- A web browser with access to the CPUlator simulator.
- The LEDS2P.s file provided with this report.
- Architecture selection: Nios II → Altera DE2-115 system.

3.2 Loading and starting the program

1. Open CPUlator and select the "Nios II" architecture and the "Altera DE2-115" system.
2. Use File ▸ Open... and select RETO2P.s, or paste its contents into the editor.
3. Press Compile and Load (F5). "Compile succeeded" should appear in the Messages panel.
4. Verify that the "JTAG UART" (Input/Output) and "Red LEDs" panels are visible; if not, enable them from the device list.
5. Press Continue (F3). The banner and menu appear (Figure 1). Click inside the terminal so it captures keyboard input.

3.3 Input conventions

- Enter is not required: each keypress is immediately processed and echoed to the screen.
- Operands are single-character hexadecimal digits: 0–9 represent 0–9; A–F (or a–f) represent 10–15.
- The continuation prompt accepts S/s (continue) or N/n (exit).
- Only keys 1–5 are accepted during the game.

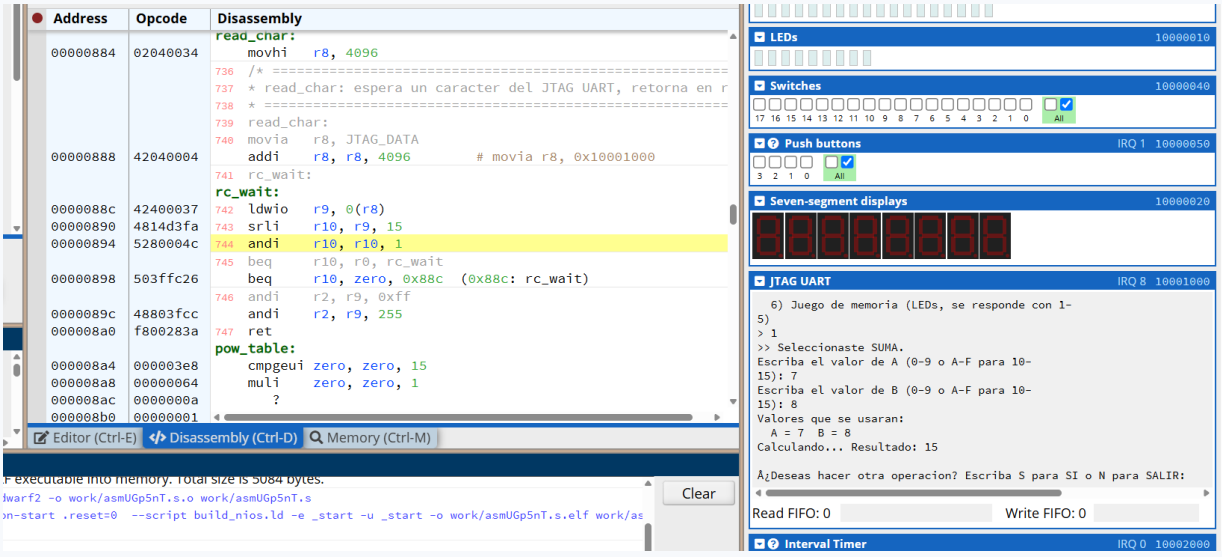
Tabla 3. Referencia rápida de teclado.

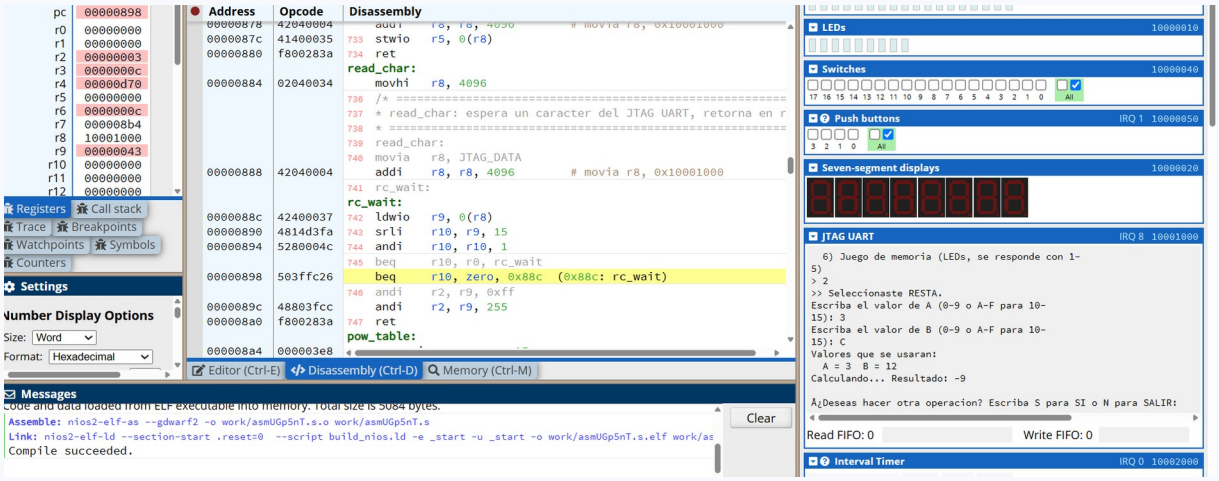
Key	Meaning	Location
0 … 6	Select operation	Main menu
0 … 9	Values 0 … 9	Operand entry
A … F	Values 10 … 15	Operand entry
S / N	Continue/ Out	After each operation
1 … 5	LED position	Memory game

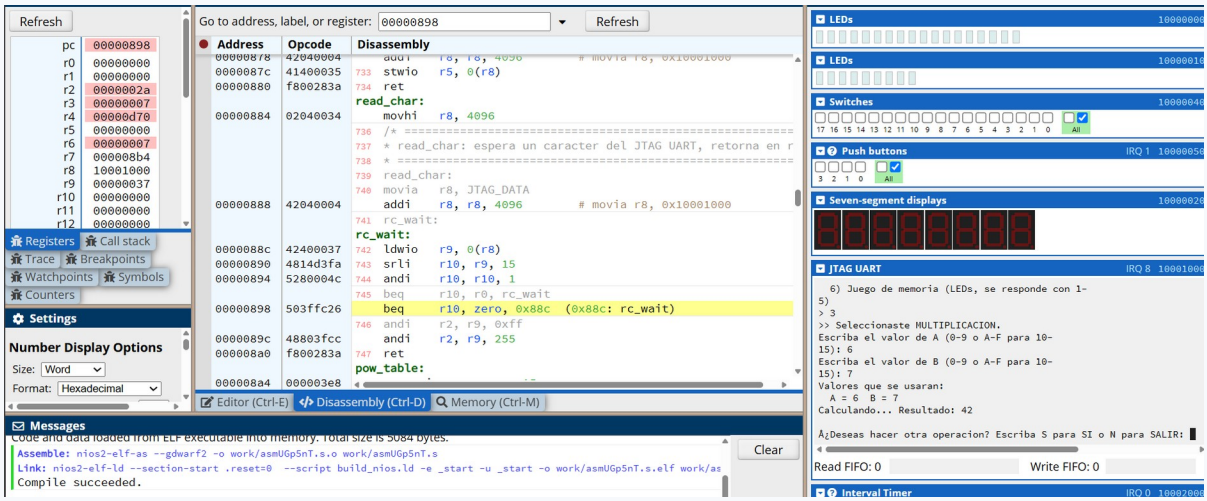
3.4 Usage of each operation

Operations 1 through 3 (addition, subtraction, and multiplication)

Select the option and enter A and B when prompted. The program displays the values to be used, pauses briefly, and prints the result. For subtraction, a negative result is displayed with its sign. Figure 4 shows three consecutive executions.

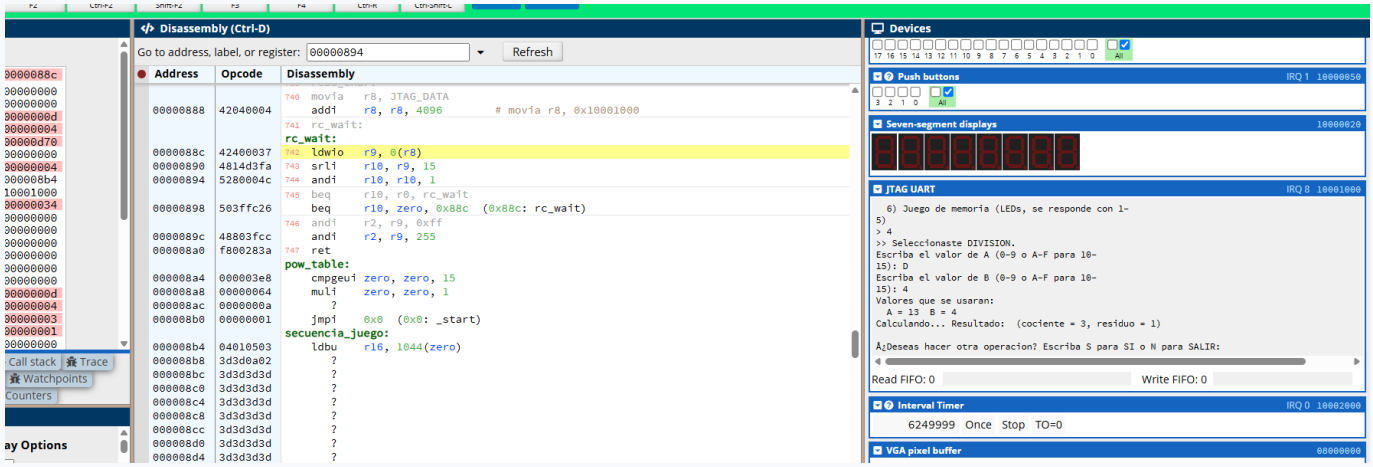

Address
Opcode
Disassembly
read_char:
00000884 02040034 movhi r8, 4096
00000888 42040004 addi r8, r8, 4096 # movia r8, 0x10001000
rc_wait:
0000088c 42400037 ldwio r9, 0(r8)
00000890 4814d3fa srli r10, r9, 15
00000894 5280004c andi r10, r10, 1
00000898 503ffc26 beq r10, zero, 0x88c (0x88c: rc_wait)
0000089c 48803fcc andi r2, r9, 255
000008a0 f800283a ret
pow_table:
000008a4 000003e8 cmpgeui zero, zero, 15
000008a8 00000064 muli zero, zero, 1
000008ac 0000000a ?
000008b0 00000001
Editor (Ctrl-E)
Disassembly (Ctrl-D)
Memory (Ctrl-M)
Clear
LEDs 10000010
Switches 10000040
Push buttons IRQ 1 10000050
Seven-segment displays 10000020
JTAG UART IRQ 8 10001000
6) Juego de memoria (LEDs, se responde con 1-5)
> 1
>> Seleccionaste SUMA.
Escriba el valor de A (0-9 o A-F para 10-15): 7
Escriba el valor de B (0-9 o A-F para 10-15): 8
Valores que se usaran:
A = 7 B = 8
Calculando... Resultado: 15
¿Deseas hacer otra operacion? Escriba S para SI o N para SALIR:
Read FIFO: 0
Write FIFO: 0
Interval Timer IRQ 0 10002000


pc 00000898
r0 00000000
r1 00000000
r2 00000003
r3 0000000c
r4 00000d70
r5 00000000
r6 0000000c
r7 000008b4
r8 10001000
r9 00000043
r10 00000000
r11 00000000
r12 00000000
Registers
Call stack
Trace
Breakpoints
Watchpoints
Symbols
Counters
Settings
Number Display Options
Size: Word
Format: Hexadecimal
Messages
Assemble: nios2-elf-as --gdwarf2 -o work/asmUGp5nT.s.o work/asmUGp5nT.s
Link: nios2-elf-ld --section-start .reset=0 --script build_nios.ld -e _start -u _start -o work/asmUGp5nT.s.elf work/as
Compile succeeded.
Clear
Write FIFO: 0


Refresh
Go to address, label, or register: 00000898
Refresh
pc 00000898
r0 00000000
r1 00000000
r2 0000002a
r3 00000007
r4 00000d70
r5 00000000
r6 00000007
r7 000008b4
r8 10001000
r9 00000037
r10 00000000
r11 00000000
r12 00000000
Registers
Call stack
Trace
Breakpoints
Watchpoints
Symbols
Counters
Settings
Number Display Options
Size: Word
Format: Hexadecimal
Messages
Assemble: nios2-elf-as --gdwarf2 -o work/asmUGp5nT.s.o work/asmUGp5nT.s
Link: nios2-elf-ld --section-start .reset=0 --script build_nios.ld -e _start -u _start -o work/asmUGp5nT.s.elf work/as
Compile succeeded.
Clear
6) Juego de memoria (LEDs, se responde con 1-5)
> 3
>> Seleccionaste MULTIPLICACION.
Escriba el valor de A (0-9 o A-F para 10-15): 6
Escriba el valor de B (0-9 o A-F para 10-15): 7
Valores que se usaran:
A = 6 B = 7
Calculando... Resultado: 42
¿Deseas hacer otra operacion? Escriba S para SI o N para SALIR:
Read FIFO: 0
Write FIFO: 0

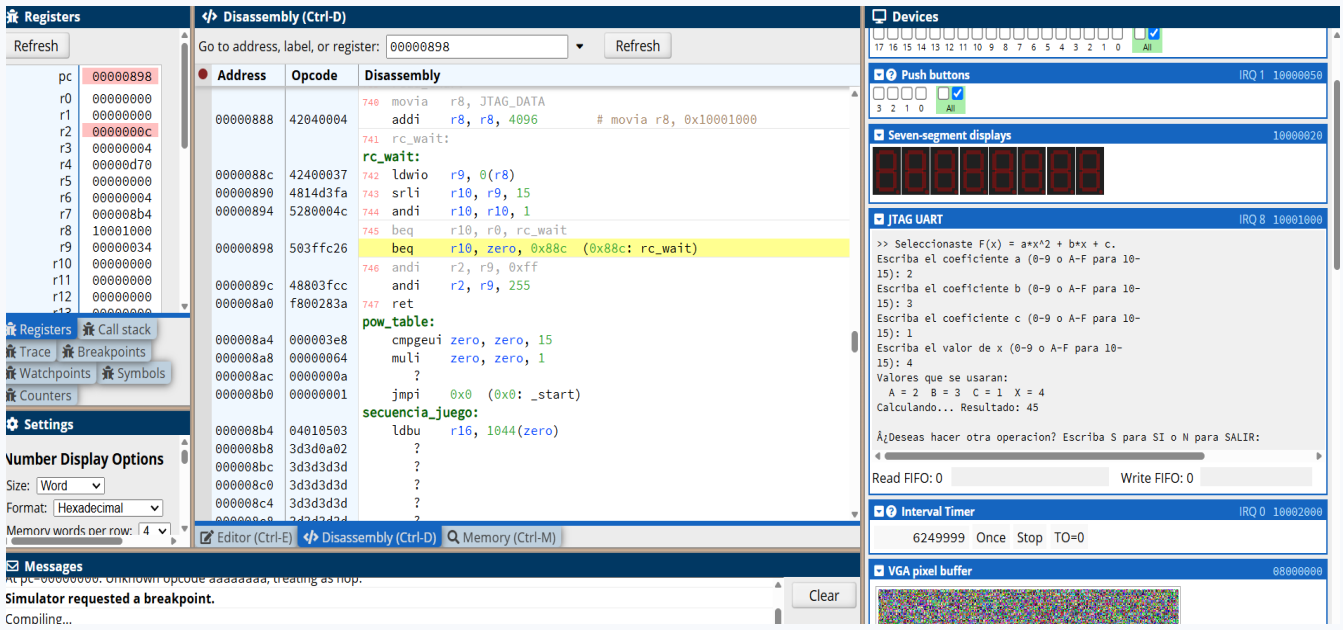
Operation 4 (integer division)

The dividend A and divisor B are requested. The result is displayed as the quotient and remainder. If B = 0, "Error: cannot divide by 0" is displayed, and the program returns to the menu.



Operation 5 (quadratic function)

The program requests the coefficients a, b, and c and the variable x—in that order—and evaluates $F(x) = a \cdot x^2 + b \cdot x + c$. The maximum possible value is $15 \cdot 15^2 + 15 \cdot 15 + 15 = 3615$, which fits within the four-digit printing routine.



Operation 6 (memory game)

6. Read the on-screen instructions and press any key when ready.
7. Observe LED0–LED4: five lights turn on one by one, slowly. LED0 (the rightmost one) is position 1, and LED4 is position 5. The terminal does not reveal which LED lit up.
8. When "Your turn!" appears, type the five positions in the order shown, pressing one key for each.
9. An incorrect key ends the game with "Incorrect sequence"; a fully correct sequence displays a congratulatory message. Keys other than 1–5 are rejected, and the same position is requested again.

3.5 Ending the session

Enter 'N' in response to the continuation prompt. The program prints a farewell message and enters an idle loop (fin_programa). To restart, press Restart (Ctrl+R in CPUlator) and then Continue.

3.6 Error messages and troubleshooting

Table 4. Troubleshooting guide.

Symptom / message	Cause and action
"Invalid option"	A key other than 1–6 was pressed in the menu. Enter a number from 1 to 6.
"Invalid value"	The key pressed is not 0–9 or A–F. Re-enter the operand.
"Invalid response"	Enter Y or N in response to the follow-up question.
"That key cannot be used"	The game only accepts keys 1–5.
Nothing happens when typing	Click inside the JTAG UART terminal to give it keyboard focus.
Simulation stops without error	CPUlator runs in bursts; press Continue once. This is simulator behavior, not a program failure.
Characters ¿ ¡ appear distorted	The terminal interprets the source's UTF-8 bytes differently; this is a cosmetic issue that does not affect operation.
Memory error on startup	Reduce STACK_TOP in the .equ section to an address within available RAM.

4. Technical description of the code

This section describes each block of LEDS2.s in the order of its appearance, following the style of a reference manual. Each subsection outlines the purpose, interface (inputs/outputs), algorithm, and relevant instructions. Visual evidence for each module is provided in the figures in Section 3.

4.1 Assembler constants and directives

```
.equ JTAG_DATA, 0x10001000
.equ JTAG_CTRL, 0x10001004
.equ LEDR,      0x10000000
.equ STACK_TOP, 0x00100000
.equ DELAY_COUNT, 150000
.equ LED_ON_DELAY,  6000000
.equ LED_OFF_DELAY, 2500000
```

The .equ directives associate symbolic names with peripheral addresses and timing parameters, such that a hardware change requires editing only a single line. DELAY_COUNT controls the pause before printing a result; LED_ON_DELAY and LED_OFF_DELAY set the duration of each LED pulse and the dark interval between pulses. Their values allow the sequence to run without any loop being long enough to trigger CPUlator's automatic pause.

4.2 Startup code

```
.org 0x0
.global _start
_start:
    movia   sp, STACK_TOP
    call    main
stop:
    br      stop
```

Execution begins at address 0x0. The stack pointer (sp, r27) is initialized using the `movia` macro-instruction, which expands into a `movhi`/`ori` pair to load a 32-bit constant. `main` is invoked via `call` (which saves the return address in `ra`), and—should it ever return—the loop at `stop` prevents the processor from executing undefined memory.

4.3 Main routine and menu

```
main:
    movia   r4, welcome_msg
    call    print_string
menu_loop:
    movia   r4, menu_msg
    call    print_string
    call    read_char
    mov     r5, r2
    call    putc
    ...
    movi    r6, '1'
    beq     r2, r6, op_suma
    ...
```

The main program prints the banner once and enters `menu_loop`. The menu is printed, a character is read (blocking), echoed back using `putc`, and compared against the ASCII codes for '1' through '6' using a sequence of `movi`/`beq` pairs that functions as a switch statement. If no match is found, `invalid_msg` is printed and the loop restarts. Each operation concludes with an unconditional jump back to `menu_loop` (Figure 1 and Figure 9).

4.4 Arithmetic modules

4.4.1 Addition and subtraction (`op_suma`, `op_resta`)

```
op_suma:
    movia   r4, sel_suma_msg
    call    print_string
    call    pedir_ab
    mov     r16, r2        /* A */
    mov     r17, r3        /* B */
    add     r18, r16, r17
    call    delay
    ...
```

The operands returned by `pedir_ab` (r2 = A, r3 = B) are copied to registers r16 and r17—which no auxiliary subroutine modifies—and the native `add` or `sub` instruction produces the result in r18. Subtraction can yield a negative value (e.g., 3 − 12 = −9); the result in two's complement is printed using `print_sdec`. The `delay` routine is called before printing to satisfy the display pause requirement (Figure 4).

4.4.2 Multiplication (`op_mult`, `multiplicar`)

```
multiplicar:
    subi sp, sp, 4
    stw  ra, 0(sp)
    movi r2, 0
    mov  r7, r5
    mov  r8, r6
mul_loop:
    beq  r8, r0, mul_done
    add  r2, r2, r7
    subi r8, r8, 1
    br   mul_loop
mul_done:
    ldw  ra, 0(sp)
    addi sp, sp, 4
    ret
```

The routine calculates r2 = r5 × r6 as r6 successive additions of r5, using r7 as the multiplicand and r8 as a countdown counter to preserve the input registers. The loop executes r6 times (maximum 15), with O(B) timing. Since the multiplication routine is reused in operation 5, this is the only point where the multiplication logic is maintained (Figure 4 and Figure 6).

4.4.3 Integer division (op_div)

```
    bne  r17, r0, div_ok
```

```
    movia r4, msg_div_cero
    ...
div_ok:
    mov  r5, r16
    movi r18, 0
div_loop:
    blt  r5, r17, div_done
    sub  r5, r5, r17
    addi r18, r18, 1
    br   div_loop
div_done:
    mov  r19, r5
```

Before dividing, the program checks the divisor and prints an error if it is zero, thereby avoiding an infinite loop. Otherwise, the dividend is copied to r5, and B is subtracted from it until r5 < B; the number of subtractions represents the quotient (r18), and the remaining value is the remainder (r19). This implements Euclidean division A = B·q + r, where 0 ≤ r < B (Figure 5).

4.4.4 Quadratic function (op_fx)

```
    mov r5, r19
    mov r6, r19
    call multiplicar
    mov r5, r16
    mov r6, r2
    call multiplicar
    mov r20, r2
    mov r5, r17
    mov r6, r19
    call multiplicar
    add r20, r20, r2
    add r20, r20, r18
```

The four coefficients are read using `read_value` into r16 (a), r17 (b), r18 (c), and r19 (x) and echoed to the display. The polynomial is evaluated as a·(x·x) + b·x + c, requiring three multiplication calls and two additions, accumulating the result in r20. The maximum intermediate value is 15·225 = 3375 and the maximum result is 3615; both are well below the 9999 limit of `print_udec` (Figure 6).

4.5 Input and output subroutines

4.5.1 `putc` and `read_char` (hardware layer)

```
putc:
    movia r8, JTAG_CTRL
pc_wait:
    ldwio r9, 0(r8)
    movhi r10, 0xffff
    and   r9, r9, r10
    beq   r9, r0, pc_wait
    movia r8, JTAG_DATA
    stwio r5, 0(r8)
    ret

read_char:
    movia r8, JTAG_DATA
rc_wait:
```

```
    ldwio r9, 0(r8)
    srli  r10, r9, 15
    andi  r10, r10, 1
    beq   r10, r0, rc_wait
    andi  r2, r9, 0xff
    ret
```

putc reads the control register and masks bits 31:16 (WSPACE); if the result is zero, the transmit FIFO is full and the routine continues polling. When space is available, the character in r5 is written to the data register. read_char polls the RVALID bit (bit 15) of the data register; when active, the low byte contains the received character, which is returned in r2. Both are leaf routines and do not require a stack frame.

4.5.2 print_string

Prints a null-terminated string whose address is in r4. The pointer is saved on the stack frame for robustness against future changes. The loop loads a byte using ldb, terminates if it is zero, and otherwise calls putc and advances the pointer.

4.5.3 read_value

```
    movi r6, '0'
    blt  r2, r6, rv_invalid
    movi r6, '9'
    bgt  r2, r6, rv_check_af
    movi r6, '0'
    sub  r2, r2, r6      /* '0'..'9' → 0..9 */
    ...
```

Reads an operand and converts the ASCII code to a binary value of 0–15. Characters '0'–'9' are converted by subtracting 0x30; 'A'–'F' and 'a'–'f' are converted by subtracting the base character value and adding 10. Any other character triggers `rv_invalid`, which prints an error and jumps to `rv_retry`, where the pointer to the original prompt (stored at 0) is reloaded. Thus, validation is transparent to the caller (Figure 9).

4.5.4 pedir_ab

Requests A and B, temporarily saves A on the stack while B is being read, prints the confirmation block ("Values to be used"), and returns r2 = A and r3 = B. Using the stack for A prevents it from being overwritten by calls within `read_value`.

4.5.5 print_dec, print_udec y print_sdec

`print_dec` prints values 0–15 (operand echo): if the value is ≥ 10, it prints '1' followed by (value − 10); otherwise, it prints a single digit. `print_udec` prints an unsigned integer up to 9999 using successive subtractions against the table `pow_table = {1000, 100, 10, 1}`; a flag in the stack frame suppresses leading zeros but always prints the last digit, ensuring that zero is displayed as "0". `print_sdec` handles the sign: for negative numbers, it prints '-' and calls `print_udec` with the two's complement negation (0 − r5). This avoids reliance on a hardware division instruction, which the Nios II core might not include.

4.5.6 delay y delay_custom

```
delay:
    movia r8, DELAY_COUNT
```

```
delay_loop:
    subi  r8, r8, 1
    bne   r8, r0, delay_loop
    ret
```

A busy-wait loop decrements a counter to zero. `delay` uses the constant `DELAY_COUNT` (150,000 iterations) for the pause preceding the result; `delay_custom` receives the number of iterations in `r5` and uses it for the LED routine. Each iteration executes two instructions, so the actual duration depends on the simulated clock and can be adjusted by editing the `.equ` values.

4.5.7 preguntar_continuar

Reads 'S'/'s' or 'N'/'n'. With 'S', it restores `ra` and returns, allowing the caller to jump to `menu_loop`. With 'N', it prints the farewell message and proceeds to `fin_programa` (an infinite self-loop). Any other character triggers a retry message and a new read operation (Figure 9).

4.6 Memory Game (`op_juego`)

4.6.1 Sequence generation

```
    slli r10, r7, 2     /* r7*4 */
    add  r10, r10, r7   /* r7*5 */
    addi r10, r10, 3    /* r7*5 + 3 */
    andi r7,  r10, 0xFF /* mod 256 */
    ...   (reducir módulo 5 y sumar 1)
```

The Nios II lacks a hardware random number source, so a linear congruential generator [7] is used: Xn+1 = (5·Xn + 3) mod 256. The seed is the ASCII code of the key the user presses to start; therefore, different keys produce different sequences. Each state is reduced modulo 5 via repeated subtraction and incremented by one to obtain positions 1–5. Multiplication by 5 is implemented as a shift and addition (x·4 + x). Examples: seed '1' (0x31) produces 4, 5, 5, 3, 2; seed 'A' produces 3, 3, 2, 4, 2.

4.6.2 LED Display

```
mostrar_led_pos:
    subi r6, r5, 1
    movi r7, 1
    sll  r8, r7, r6      /* máscara = 1 << (pos-1) */
    movia r9, LEDR
    stwio r8, 0(r9)      /* LED encendido */
    movia r5, LED_ON_DELAY
    call  delay_custom
    stwio r0, 0(r9)      /* LED apagado */
    movia r5, LED_OFF_DELAY
    call  delay_custom
```

The position (1–5) is converted into a single-bit mask by shifting a '1' to the left by (position − 1) places and writing it to the LEDR register. After LED_ON_DELAY iterations, the register is cleared, followed by a dark interval of LED_OFF_DELAY iterations, ensuring that two consecutive identical positions remain distinguishable (Figure 7).

4.6.3 Response Verification

The stored sequence is traversed using two registers (r18 as a pointer and r19 as a remaining-count counter). For each element, `leer_posicion_juego` reads the user's key input, maps '1'–'5' to the values 1–5, and rejects other keys (repeating the same position). If the number differs from the stored value, the game ends immediately with `juego_fallo_msg`; if all five match, `juego_gano_msg` is displayed (Figure 8).

```
los numeros del 1 al 5 (el mismo numero de cada posicion).
Escribe la siguiente posicion (1, 2, 3, 4 o 5)... 3
Escribiste la posicion: 3
Escribe la siguiente posicion (1, 2, 3, 4 o 5)... 5
Escribiste la posicion: 5
Escribe la siguiente posicion (1, 2, 3, 4 o 5)... 2
Escribiste la posicion: 2
Escribe la siguiente posicion (1, 2, 3, 4 o 5)... 2
Escribiste la posicion: 2
```

4.7 Data section

The .align 2 directive ensures that pow_table is word-aligned, a requirement for ldw. The table is followed by secuencia_juego (5 bytes, overwritten at runtime) and all null-terminated messages declared with .asciz. The visible text is in Spanish, and the labels follow consistent naming conventions (sel_*_msg, prompt_*, juego_*_msg).

4.8 Register usage and calling convention

Table 5. Register usage conventions.

Register	Alias / Role	Usage in this program
r0	zero	Constant 0 (comparisons, clearing LEDR)
r2, r3	Return values	r2: result of read_char, read_value, multiplicar, leer_posicion_juego; r3: B from pedir_ab
r4	Argument	String pointer for print_string / read_value
r5, r6	Arguments	r5: character or number to print / first factor; r6: second factor / temporary
r7–r12	Temporaries	Loop counters, masks, comparison constants (modified by subroutines)
r16–r20	Saved values	Operands and result of the current operation (not modified by helper routines)
r27	sp	Stack pointer, initialized to STACK_TOP
r31	ra	Return address stored by call

Non-leaf routines set up a stack frame with `subi sp, sp, n` upon entry, save `ra`, and restore it with `addi sp, sp, n` before the `ret` instruction. Leaf routines (`putc`, `read_char`, `delay`, `delay_custom`) omit the frame.

5. Testing and performance evidence

5.1 Testing and performance evidence

The validation combines (i) functional black-box testing of each menu option using typical, boundary, and invalid inputs, and (ii) analytical verification: the expected value for each test was manually derived from the mathematical definition of the operation prior to execution and compared against the output displayed on the JTAG UART terminal. Each test references the supporting CPUlator screenshot. Boundary values (0 and 15) exercise the limits of the 4-bit operand range and the extremes of the printing routines.

5.2 Test cases with evidence

Table 6. Test plan with analytically derived expected results.

ID	Module	Input	Expected result	Evidence
T1	Addition	A=7, B=8	15	Fig. 4
T2	Subtraction	A=3, B=C (12)	−9	Fig. 4
T3	Multiplication	A=6, B=7	42	Fig. 4
T4	Division	A=D (13), B=4	quotient 3, remainder 1	Fig. 5
T5	Division	A=5, B=0	Error: division by zero	Fig. 5
T6	F(x)	a=2, b=3, c=1, x=4	45	Fig. 6
T7	F(x)	a=F, b=F, c=F, x=F	3615	Fig. 6
T8	Game	Tecla '1'; answers 4,5,5,3,2	Victory message	Figs. 7 y 8
T9	Game	Tecla '1'; first answer 1	Failure message	Fig. 8
T10	Validation	Menu: 9	"Invalid option" and menu again	Fig. 9
T11	Validation	Operand: Z, then b	Rejection and acceptance as 11	Fig. 9
T12	Exit	Answer N	Farewell; program idle	Fig. 9

Analytical derivations: T2: 3 − 12 = −9; T3: 6·7 = 42; T4: 13 = 4·3 + 1; T6: 2·16 + 3·4 + 1 = 45; T7: 15·225 + 15·15 + 15 = 3375 + 225 + 15 = 3615. For the game, the sequence with seed '1' is obtained from the recurrence X → (5X + 3) mod 256 starting at X = 49: 248, 219, 74, 117, 76, whose remainders modulo 5 plus one are 4, 5, 5, 3, 2.

5.3 Additional boundary values

The following cases are verified analytically based on the code logic and can be reproduced using the same procedure as in Section 3.

Table 7. Analytically verified boundary cases.

Input	Calculation	Expected result
Addition A=F, B=F	15 + 15	30

Input	Calculation	Expected result
Subtraction A=9, B=4	9 − 4	5
Multiplication A=F, B=F	15 × 15	225
Multiplication A=0, B=B	0 × 11	0
Division A=F, B=1	15 = 1·15 + 0	quotient 15, remainder 0
Division A=2, B=9	2 = 9·0 + 2	quotient 0, remainder 2
F(x): a=0, b=5, c=7, x=3	0 + 15 + 7	22
F(x): a=1, b=0, c=0, x=1	1·1	1

5.4 Performance analysis

Table 8. Time complexity of main routines.

Routine	Complexity	Comment
multiplicar	O(B), ≤ 15 iterations	Each iteration: 4 instructions (beq, add, subi, br)
Bucle de op_div	O(A/B), ≤ 15 iterations	Worst caseB = 1, A = 15
print_udec	4 positions × ≤ 9 restas	Bounded; constant time in practice
op_fx	3 multiplications + 2 additions	≤ 45 internal multiply iterations
delay	150 000 iterations	Pausa fija antes de cada resultado
Game presentation	5 × (6 000 000 + 2 500 000) iterations	Deliberately slow to allow for memorization

All numerical routines are constrained by the 4-bit operand range; consequently, execution time is dominated by intentional delays and UART polling—that is, by the user's speed. The maximum numerical value (3615) and the maximum intermediate value (3375) fall within the 0–9999 range of `print_udec`, so overflow is impossible with any valid input.

6. Limitations and future work

- Optional 7-segment display output (using the additional decimal point) is not implemented; the result appears only on the UART terminal. This could be added by writing BCD-to-7-segment patterns to the DE2-115's HEX registers.
- Operands are limited to the 0–15 range due to the single-key input design; multi-digit input would require buffered parsing.
- `print_udec` supports values up to 9999; products of larger operands would require a more extensive lookup table.
- Polling blocks the processor while it waits for input; an interrupt-driven design would free it for other tasks.

- The pseudo-random number generator is seeded using the start key, meaning the same key press reproduces the same sequence; sampling a free-running counter upon the key press would improve unpredictability.

7. Conclusions

A modular calculator and a memory game were successfully implemented in Nios II assembly language. The design demonstrates memory-mapped I/O using polling, stack-based subroutine linking, ASCII-to-binary conversion, software implementations of multiplication, division, and pseudo-random number generation, and precise control of an I/O peripheral (LEDR). Robust input validation and the continue/exit loop make the program usable without author intervention, while the analytical test plan provides objective evidence of correctness using typical, boundary, and erroneous inputs.

References

Intel Corporation, “Nios II Processor Reference Guide,” Intel FPGA Documentation, San Jose, CA, EE. UU.

Intel Corporation, “Embedded Peripherals IP User Guide: JTAG UART Core,” Intel FPGA Documentation, San Jose, CA, EE. UU.

Terasic Technologies, “DE2-115 User Manual,” Hsinchu, Taiwán.

H. Wong, “CPUlator Computer System Simulator.” [En línea]. Disponible: https://cpulator.01xz.net/

Intel FPGA University Program, “Introduction to the Intel Nios II Soft Processor,” tutorial, Intel Corporation.

D. A. Patterson y J. L. Hennessy, Computer Organization and Design: The Hardware/Software Interface, 5.ª ed. Waltham, MA, EE. UU.: Morgan Kaufmann, 2014.

D. E. Knuth, The Art of Computer Programming, Vol. 2: Seminumerical Algorithms, 3.ª ed. Reading, MA, EE. UU.: Addison-Wesley, 1997.